ATESTool

Installation and User Guide

Wouter Deleersnyder

October 25, 2026

Abstract

This guide explains how to install ATESTool from its public demonstration repository, verify the installation, and run the two examples. The examples introduce interference and sensitivity to well placement,. No prior knowledge of the package internals is required.

Contents

1 What ATESTool does **3**

2 Installation **3**
2.1 Prerequisites . . . 3
2.2 Step 0: Check the installed Python version . . . 3
2.3 Step 1: Obtain the source code . . . 3
2.4 Step 2: Create an isolated Python environment . . . 4
2.5 Step 3: Install ATESTool and the mesh backend . . . 4
2.6 Step 4: Install MODFLOW 6 . . . 4
2.7 Step 5: Verify the installation . . . 5

3 Exercise 1: Run and inspect one model **5**
3.1 Learning objectives . . . 6
3.2 Task 1: inspect the configuration . . . 6
3.3 Task 2: run the model and create a report . . . 6
3.4 Task 3: inspect the output . . . 6

4 Exercise 2: Case 1—reciprocal interaction **7**
4.1 Question . . . 7
4.2 Task 1: run the three states . . . 7
4.3 Task 2: construct the comparison in Excel . . . 7
4.4 Task 3: interpret the reciprocal effects . . . 7

5 Exercise 3: Case 2—layout sensitivity **8**
5.1 Question . . . 8
5.2 Task 1: run the reference and four layouts . . . 8
5.3 Task 2: construct the comparison in Excel . . . 8
5.4 Task 3: interpret the layout effects . . . 8

6 Recommended workflow for new cases **9**

7 Troubleshooting **9**

8 Further documentation **10**

A Installing a Suitable Python Version **11**
A.1 Windows . . . 11
A.2 macOS . . . 11
A.2.1 Option A: official Python installer . . . 12
A.2.2 Option B: Homebrew . . . 12
A.3 Ubuntu and Debian-based Linux distributions . . . 12
A.4 Other Linux distributions . . . 13
A.5 Installing Python with `pyenv` . . . 13
A.6 Final verification . . . 14

1 What ATESTool does

ATESTool is a configuration-driven tool for modelling Aquifer Thermal Energy Storage (ATES) systems and their interaction with groundwater extraction. It creates an unstructured mesh, builds coupled MODFLOW 6 groundwater-flow (GWF) and energy-transport (GWE) models through FloPy, runs the model, and writes processed results.

The public repository is available at:

https://github.com/WouterDls/ATESTool-demo

The repository has three main directories:

`ATESTool/` Python source code.

`docs/` Technical and API documentation.

`examples/` The three examples.

2 Installation

2.1 Prerequisites

Before installing ATESTool, ensure that the following are available:

- Python 3.10 or newer;
- Git (optional: the repository can also be downloaded as a ZIP file);
- MODFLOW 6 version 6.5 or newer (Step 4 explains how to install it).

The optional Triangle package produces the intended refined Voronoi mesh and is installed automatically in Step 3. Without it, ATESTool falls back to SciPy's Delaunay triangulation, which does not provide the same per-well refinement.

2.2 Step 0: Check the installed Python version

Open a terminal and try:

```
python3 --version
```

On Windows, use:

```
py --version
```

The reported version must be Python 3.10 or newer. For example, Python 3.10, 3.11, 3.12, or 3.13 is suitable. If the command is not found or reports an older version, install a newer Python interpreter using the instructions in Appendix A.

2.3 Step 1: Obtain the source code

Open a terminal and clone the repository:

```
git clone https://github.com/WouterDls/ATESTool-demo.git
cd ATESTool-demo
```

Alternatively, download the repository as a ZIP file from GitHub, extract it, and open a terminal in the extracted `ATESTool-demo` directory.

2.4 Step 2: Create an isolated Python environment

Using a virtual environment prevents ATESTool dependencies from changing other Python projects. A virtual environment uses the Python interpreter that was specified during the creation. Therefore, first identify an installed Python 3.10 or newer and then use that interpreter explicitly.

Step 2a: Find a suitable interpreter

On macOS and Linux, list the available Python interpreters:

```
which -a python3 python3.10 python3.11 python3.12 python3.13
```

On Windows PowerShell, list the installed Python interpreters and their paths:

```
py -0p
```

Note the name or version of an interpreter that is 3.10 or newer.

Step 2b: Create and activate the environment

On macOS and Linux, replace `python3.12` below with the interpreter found in Step 2a (for example `python3` if that already reports 3.10 or newer):

```
python3.12 -m venv .venv
source .venv/bin/activate
```

On Windows PowerShell, replace `3.12` with the version found in Step 2a:

```
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
```

If PowerShell refuses to run `Activate.ps1` because script execution is disabled, see the execution-policy note in Appendix A.1.

After activation, the terminal prompt will normally begin with `(.venv)`. Upgrade the packaging tools:

```
python -m pip install --upgrade pip setuptools wheel
```

2.5 Step 3: Install ATESTool and the mesh backend

Install the package in editable mode with the Triangle mesh backend:

```
python -m pip install -e ".[mesh]"
```

To install without Triangle (and accept the SciPy fallback mesh), use `python -m pip install -e .` instead.

2.6 Step 4: Install MODFLOW 6

ATESTool needs a MODFLOW 6 executable to run models. Choose one of the options below.

Option A (recommended): install with FloPy's `get-modflow` FloPy is installed together with ATESTool and includes a `get-modflow` command that downloads the correct MODFLOW 6 build for the current platform, including Apple Silicon. With the virtual environment active, run:

```
get-modflow :python
```

This places `mf6` in the virtual environment's `bin` (or `Scripts`) directory, which is on `PATH` whenever the environment is active. An internet connection is required.

Option B: download an official release Download the appropriate release from https://github.com/MODFLOW-ORG/modflow6/releases. Extract it and place the `mf6` executable (or `mf6.exe` on Windows) in a directory on `PATH`, or use Option C. Apple Silicon users should select the ARM64 build.

On macOS, a downloaded executable may be blocked the first time it runs. If so, remove the quarantine flag:

```
xattr -d com.apple.quarantine /absolute/path/to/mf6
```

Option C: point ATESTool directly to the executable If the executable is not on `PATH`, define `MF6_BIN` before running ATESTool.

On macOS or Linux:

```
export MF6_BIN="/absolute/path/to/mf6"
```

On Windows PowerShell:

```
$env:MF6_BIN = "C:\absolute\path\to\mf6.exe"
```

These settings last only for the current terminal session.

2.7 Step 5: Verify the installation

Check that Python can import the package:

```
python -c "import ATESTool; print(ATESTool.__version__)"
```

Verify the command-line interface:

```
ATESTool --help
```

Verify that MODFLOW 6 can be found and reports version 6.5 or newer:

```
mf6 --version
```

If you use `MF6_BIN` instead, run `"$MF6_BIN" -version` on macOS or Linux, or `& $env:MF6_BIN -version` in PowerShell.

3 Exercise 1: Run and inspect one model

This exercise introduces the YAML-first workflow.

3.1 Learning objectives

After this exercise, you should be able to run a YAML configuration, identify the generated files, and use the HTML report to check the model input and principal results.

3.2 Task 1: inspect the configuration

Open `examples/single_run.yaml`. Identify the following blocks before running it:

`mesh` Domain dimensions, background cell size, and refinement rings.

`model.aquifer` Hydraulic and thermal properties of the aquifer layer.

`model.chd_heads` Regional hydraulic-head boundary conditions.

`model.doublets` ATES well locations, flow rate, and injection temperatures.

`model.extraction_well` Position and pumping rate of the extraction well.

`model.simulation` Duration, seasonal length, and time stepping.

`analysis` Requested output variables and reporting times.

Coordinates are metres, time is days, flow is cubic metres per day, and temperature is degrees Celsius.

3.3 Task 2: run the model and create a report

From the repository root, run:

```
ATESTool examples/single_run.yaml \
  --workspace examples/output/single_run \
  --report
```

Using a separate workspace is important: it prevents the files from one configuration from being mixed with those of another. The `-report` option creates a compact HTML report from the completed run without launching extra MODFLOW simulations.

3.4 Task 3: inspect the output

Open `examples/output/single_run/report.html`. Locate the model layout, regional flow, well-centred drawdown, temperature results, extraction-well time series, energy balance, and recovery efficiency. Then find these reusable data files in the same folder:

`timeseries.csv` Extraction-well temperature and head through time.

`energy.csv` Seasonal energy injected, extracted, and recovered.

`recovery.csv` Recovery efficiencies arranged by ATES well.

`stats.json` Summary statistics for the run.

Exercise questions Record the simulated duration, the direction of regional groundwater flow, the maximum drawdown close to the extraction well, and the final extraction temperature. Explain which figure or table you used for each answer.

4 Exercise 2: Case 1—reciprocal interaction

4.1 Question

How does a balanced ATES system change the temperature of extracted groundwater, and how does groundwater extraction in turn change ATES recovery? Answering both parts requires three matched physical states.

4.2 Task 1: run the three states

Run each YAML file independently from the repository root:

```
ATESTool examples/case1/combined.yaml \
  --workspace examples/output/case1/combined --report
ATESTool examples/case1/extraction_only.yaml \
  --workspace examples/output/case1/extraction_only --report
ATESTool examples/case1/ates_only.yaml \
  --workspace examples/output/case1/ates_only --report
```

The states are:

`combined` ATES and groundwater extraction are active.

`extraction_only` Extraction is active and ATES flow is zero.

`ates_only` ATES is active and groundwater extraction is zero.

All three configurations use the same mesh settings and mesh-only refinement points, so their results can be compared cell for cell. Each workspace contains its own HTML report and CSV outputs. Inspect the three reports before aggregating any data.

4.3 Task 2: construct the comparison in Excel

Open `examples/case1/case1_analysis_template.xlsx` and read its `Guide` sheet. Paste *values only*, including the header row, at cell A1 of each yellow source sheet:

Generated file	Excel source sheet
`combined/timeseries.csv`	`Combined time`
`extraction_only/timeseries.csv`	`Extraction time`
`combined/energy.csv`	`Combined energy`
`ates_only/energy.csv`	`ATES energy`

Do not paste into `Analysis`, `Recovery`, or `Summary`; those sheets contain formulas. Save your completed workbook under a new name.

4.4 Task 3: interpret the reciprocal effects

Use the `Analysis` and `Summary` sheets to determine the maximum ATES-induced warming and cooling at the extraction well. Use `Recovery` to compare mean ATES recovery with and without groundwater extraction.

Exercise questions Which comparison isolates the effect of ATES on extraction temperature? Which comparison isolates the effect of extraction on ATES recovery? Report the magnitude and sign of both effects and explain what the sign means physically.

After completing the exercise, compare your workbook with `examples/case1/case1_analysis_solution.xlsx`. The solution is a checking aid; do not use it as the source for your own analysis.

5 Exercise 3: Case 2—layout sensitivity

5.1 Question

How do distance, orientation, and warm/cold-well ordering change the interference signal when aquifer properties and operating rates remain fixed?

5.2 Task 1: run the reference and four layouts

```
ATESTool examples/case2/without_ates.yaml \
  --workspace examples/output/case2/without_ates --report
ATESTool examples/case2/near_downstream.yaml \
  --workspace examples/output/case2/near_downstream --report
ATESTool examples/case2/far_downstream.yaml \
  --workspace examples/output/case2/far_downstream --report
ATESTool examples/case2/transverse.yaml \
  --workspace examples/output/case2/transverse --report
ATESTool examples/case2/reversed_longitudinal.yaml \
  --workspace examples/output/case2/reversed_longitudinal --report
```

`without_ates` Common groundwater-extraction reference.

`near_downstream` Longitudinal doublet close to extraction.

`far_downstream` The same layout farther downstream.

`transverse` Doublet perpendicular to regional flow.

`reversed_longitudinal` Longitudinal warm/cold ordering reversed.

Open every `report.html`. Confirm that the well layouts match their descriptions and compare the individual extraction-temperature histories.

5.3 Task 2: construct the comparison in Excel

Open `examples/case2/case2_analysis_template.xlsx`. For each run, copy its complete `timeseries.csv`, including the header, and paste values only at A1 of the matching yellow sheet: `Without ATES`, `Near time`, `Far time`, `Transverse time`, or `Reversed time`. The formulas subtract the common reference and update the chart and summary automatically.

5.4 Task 3: interpret the layout effects

Use the calculated histories and summary metrics to answer:

1. Does moving the doublet farther away reduce the maximum interference signal?

2. Does the transverse layout cause warm and cold signals to partly cancel?

3. Does reversing the longitudinal ordering change the sign of the response?

4. Which layout has the largest absolute temperature change, and which has the largest final-year mean change?

5. Why is comparison with `without_ates` preferable to comparison with the ambient temperature alone?

Save your completed workbook under a new name. Then compare it with `examples/case2/case2_analysis_sol`. Use any disagreement to check that the complete CSV files were pasted into the correct sheets and that the header rows begin at A1.

6 Recommended workflow for new cases

Use the following sequence when adapting the examples:

1. Copy `examples/base.yaml`
2. Replace the aquifer properties.
3. Set the regional boundary heads and verify the implied flow direction.
4. Define extraction and ATES well coordinates and rates.
5. Start with a shorter simulation for setup checks, and run it in its own workspace:

```
ATESTool my_case.yaml --workspace output/my_case
```

6. Optional: Review the mesh-quality, Péclet, thermal-resolution, and Courant numbers logged during the run. For a full verification bundle, add `-diagnostic`; results are written to `<workspace>/diagnostic/`, with an overview in `summary.txt`. This costs roughly twelve baseline runs.
7. Optional: Refine space and time where the decision variable is sensitive.
8. Analyse complete time series as well as summary metrics.

7 Troubleshooting

`ModuleNotFoundError: ATESTool` Activate the virtual environment and rerun `python -m pip install -e ".[mesh]"` from the repository root.

`Activate.ps1` cannot be loaded (Windows) Script execution is disabled. See the execution-policy note in Appendix A.1.

MODFLOW executable not found Run `get-modflow :python` with the environment active, put `mf6` on `PATH`, or set `MF6_BIN` to its absolute path.

GWE package or input is unsupported Confirm with `mf6 -version` that MODFLOW 6 is version 6.5 or newer.

Triangle is unavailable Install `triangle` or reinstall with `.[mesh]`. Review Triangle's licence before commercial use.

The simulation is slow Reduce the number of simulated years while checking setup, but restore the intended duration for analysis. Mesh refinement, time steps, and diagnostic studies all increase runtime.

Mesh, Péclet, thermal-resolution, or Courant warnings Treat these as accuracy advisories. Tighten spatial refinement or time stepping and compare the decision variable between successive levels.

An output directory already exists Choose a new workspace or deliberately remove old generated output before rerunning. Keeping separate workspaces prevents accidental mixing of results.

8 Further documentation

The repository's `README.md` provides a concise overview. More detailed configuration, meshing, CLI, and API information is available at https://wouterdls.github.io/ATESTool-demo/.

A Installing a Suitable Python Version

ATESTool requires Python 3.10 or newer. The Python interpreter must be installed *before* the virtual environment is created. A virtual environment does not install a new version of Python; it uses the interpreter that created it.

A.1 Windows

1. Open the official Python download page:

   https://www.python.org/downloads/windows/

2. Download a current 64-bit Python installer. Python 3.12 is a suitable choice for ATESTool. Recent releases may offer the *Python install manager* instead of the classic installer; with the install manager, run `py install 3.12` in PowerShell and continue at the `py -0p` step below.

3. Start the installer.

4. On the first installation screen, enable the option **Add Python to PATH**.

5. Select **Install Now** and complete the installation.

6. Open a new PowerShell window and list the installed Python interpreters:

```
py -0p
```

7. Verify the selected version:

```
py -3.12 --version
```

8. Create the ATESTool virtual environment with that specific interpreter:

```
cd path\to\ATESTool-demo
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
python --version
```

If PowerShell prevents activation because script execution is disabled, allow locally created scripts for the current user:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Close and reopen PowerShell, return to the repository directory, and activate the environment again. Changing the execution policy may be restricted on managed computers; contact the system administrator when necessary.

A.2 macOS

Python can be installed using the official installer or Homebrew. Do not replace or modify the Python interpreter supplied internally by macOS.

A.2.1 Option A: official Python installer

1. Open:

   https://www.python.org/downloads/macos/

2. Download and run a current universal macOS installer.

3. Open a new Terminal window and verify the installation:

```
python3.12 --version
```

4. Create and activate the environment:

```
cd /path/to/ATESTool-demo
python3.12 -m venv .venv
source .venv/bin/activate
python --version
```

A.2.2 Option B: Homebrew

If Homebrew is already installed, run:

```
brew install python@3.12
python3.12 --version
```

Then create and activate the environment:

```
cd /path/to/ATESTool-demo
python3.12 -m venv .venv
source .venv/bin/activate
python --version
```

If Homebrew is not installed, either use the official Python installer above or follow the installation instructions at https://brew.sh/.

A.3 Ubuntu and Debian-based Linux distributions

First inspect the Python versions provided by the distribution:

```
python3 --version
apt-cache policy python3
```

If the default interpreter is Python 3.10 or newer, install the virtual-environment support package:

```
sudo apt update
sudo apt install python3-venv
```

Then create the environment:

```
cd /path/to/ATESTool-demo
python3 -m venv .venv
source .venv/bin/activate
python --version
```

If a suitable version is available under a version-specific package name, it can be installed explicitly. For example:

```
sudo apt update
sudo apt install python3.12 python3.12-venv
python3.12 --version
python3.12 -m venv .venv
source .venv/bin/activate
python --version
```

Not every Ubuntu or Debian release provides `python3.12` in its standard repositories. Do not blindly add an unofficial package repository. On older systems, use a newer supported operating-system release, an administrator-provided Python installation, or `pyenv` as described below.

A.4 Other Linux distributions

Use the distribution's package manager to install Python 3.10 or newer together with the packages needed to create virtual environments.

For example, on Fedora:

```
sudo dnf install python3 python3-pip
python3 --version
```

On Arch Linux:

```
sudo pacman -S python python-pip
python --version
```

After confirming that the version is suitable:

```
cd /path/to/ATESTool-demo
python3 -m venv .venv
source .venv/bin/activate
python --version
```

A.5 Installing Python with `pyenv`

The `pyenv` tool is useful on macOS and Linux when the operating system provides an older Python version or when the user does not have permission to change the system Python. It installs an additional interpreter under the user's account. (On Windows, the separate `pyenv-win` project provides similar functionality.)

Installation instructions are available at:

https://github.com/pyenv/pyenv

After installing `pyenv`, install and select Python 3.12:

```
pyenv install 3.12
cd /path/to/ATESTool-demo
pyenv local 3.12
python --version
python -m venv .venv
source .venv/bin/activate
python --version
```

The command `pyenv install 3.12` selects the newest available Python 3.12 patch release. If the installed `pyenv` version requires a complete version number, list the available releases first:

```
pyenv install --list
```

Then install one of the listed Python 3.12 releases.

A.6 Final verification

After activating the virtual environment, verify both the interpreter and its location.

On macOS or Linux:

```
python --version
which python
```

On Windows PowerShell:

```
python --version
Get-Command python
```

The reported Python version must be 3.10 or newer. The interpreter path should point inside the repository's `.venv` directory.

Upgrade the packaging tools before installing ATESTool:

```
python -m pip install --upgrade pip setuptools wheel
```

Finally, install ATESTool and the Triangle mesh backend:

```
python -m pip install -e ".[mesh]"
```

If an interpreter older than Python 3.10 is used, `pip` should reject the ATESTool installation because the package declares:

```
requires-python = ">=3.10"
```

This package check is a final safeguard. Selecting and verifying the correct interpreter before creating the virtual environment produces a clearer setup process and avoids recreating the environment later.